

- Memory lower bounds for mixed precision matrix multiplication
- An algorithm achieving the lower bound
- Connecting to dense-times random matrix multiplication
- Generalizing our lower bound via HBL

Tiling for Asymmetric Access Matrix Multiplication

- Consider the following three problems of the form $C = AB$:
 - B is an i.i.d. random matrix (random projection, JL-style)
 - B is a lower-precision matrix than A (low precision neural network training)
 - C has either a higher or lower precision than either A or B
- Suppose, in the first example above, that it is faster to recompute an entry of B instead of loading it from memory.
- Assume a single level of blocking. All three problems have a common feature: the cost to move an entry of B into the lowest level of memory is less than the cost to move an entry of A or C .
- Suppose each memory move of an entry of A or C costs a single unit, and suppose that moving an entry of B costs ρ , $0 \leq \rho < 1$, in either problem setting.

Lower Bounds for Mixed Precision Matmul

- Want memory lower bounds for both of the examples on the previous slide.
- The second example (mixed precision matmul) is slightly easier to derive a lower bound for, so we'll start there.
- A lower bound for the first example is a little harder (but perhaps possible), since it mixes accounting for computation and memory movement.

Lower Bounds for Mixed Precision Matmul

- Our computational cuboid has dimensions $m \times n \times k$ corresponding to the matrix dimensions. Split the computation into phases where each phase uses at most M words, with M as the size of fast-memory.
- Upper-bound the useful work V that can be done with M words with discrete Loomis Whitney inequality.
 - Let A_1, A_2, A_3 be the projections of the volume element V onto the faces of the computational cube):

$$|V| \leq (A_1 A_2 A_3)^{1/2}$$

- So far, the same as regular standard matmul. For the second example, the difference is the memory inequality: we constrain

$$A_1 + \rho A_2 + A_3 \leq M$$

since every entry of B costs only ρ .

Lower Bounds for Mixed Precision Matmul

- Now apply AM-GM and combine with the memory bound:

$$\sqrt[3]{\rho A_1 A_2 A_3} \leq \frac{A_1 + \rho A_2 + A_3}{3} \leq \frac{M}{3}$$

- This yields

$$(A_1 A_2 A_3)^{1/2} \leq O\left(\frac{M^{3/2}}{\sqrt{\rho}}\right)$$

$$|V| \leq O\left(\frac{M^{3/2}}{\sqrt{\rho}}\right)$$

- To upper-bound the number of stages S , we divide mnk by the upper-bound on V to get

$$S \geq O\left(\frac{mnk\sqrt{\rho}}{M^{3/2}}\right)$$

Lower Bounds for Mixed Precision Matmul

- Each stage moves M words, so multiply the number of stages by M to get the final movement cost

$$\text{words moved} \geq O\left(\frac{mnk\sqrt{\rho}}{\sqrt{M}}\right)$$

- Notice that the total number of words moved is asymptotically smaller by a factor of $\sqrt{\rho}$ compared to standard matrix multiplication.

An Algorithm that Achieves the Lower Bound

- We will keep exactly one of the three matrices stationary in memory and stream in subcolumns and subrows, respectively, of A and B , each contributing an outer product. Only a single subrow of A and B are resident in memory at any given time.

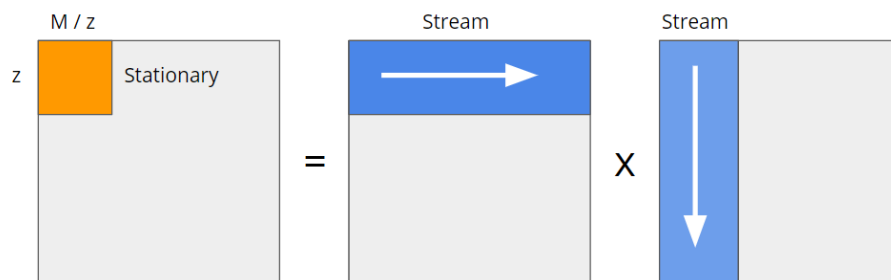


Figure 1: Tiling Scheme

An Algorithm that Achieves the Lower Bound

- Each orange block is loaded into memory exactly once. The number of orange tiles is $m/z * \frac{n}{M/z} = \frac{mn}{M}$.
- Every time an orange tile is loaded in, we stream in a block of rows from A (by subcolumns) and a block of columns from B (by subrows). These move zk and $kM\rho/z$ words, respectively
- Ignoring smaller terms, the total memory movement cost is the number of orange blocks multiplied by the number of words moved per block, which is

$$\frac{mn}{M} \left(zk + \frac{\rho kM}{z} \right) = \frac{mnk}{M} \left(z + \frac{\rho M}{z} \right)$$

- The value of z that optimizes this expression is $\sqrt{M\rho}$

An Algorithm that Achieves the Lower Bound

- The value of z that optimizes this expression is $z = \sqrt{\rho M}$, and the value of the optimum is

$$\frac{2mnk\sqrt{\rho}}{\sqrt{M}}$$

- Let β be the DRAM bandwidth; then the time taken is

$$\frac{2mnk\sqrt{\rho}}{\beta\sqrt{M}}$$

- Reasonableness check: when $\rho = 1$, then this is just the cost of standard matrix multiplication
- When $\rho = 0$, the cubic term $2mnk$ disappears entirely.
- Intuition: it costs less to load in matrix B , so we reload into memory more frequently to reduce the number of words of A we need to move.

The Connection to Dense-Times Random

- The same algorithm achieves the lower bound for dense-times random. Refer to the picture on slide 6: instead of loading in a subrow of B from memory, we will recompute it on-the-fly.
- For each of $\frac{mn}{M}$ orange tiles, we stream zk words of A from memory, and we have to recompute $\frac{kM}{z}$ words of B .
- Now define

$$\rho = \frac{\text{Time to recompute entry of } B}{\text{Time to load an entry of } A \text{ from memory}}$$

which is almost identical to the previous definition of ρ .

The Connection to Dense-Times Random

- The total time taken, including terms for memory movement and recomputation, is

$$\frac{mn}{M} \left(\frac{zk}{\beta} + \frac{\rho kM}{\beta z} \right) = \frac{mnk}{M\beta} \left(z + \frac{M\rho}{z} \right)$$

which is the same optimization problem as before, except with a different value of ρ .

A Comparison to Using Square Tiles

- What if we used square tiles like standard matrix multiplication? If $z = \sqrt{M}$ (corresponding to a square tile), the total words moved would instead be

$$\frac{mnk}{\sqrt{M}}(1 + \rho)$$

and ρ becomes an additive, not a multiplicative, factor.

- The difference between the value for a square tile and the optimal value is

$$\frac{mnk}{\sqrt{M}}(1 + \rho - 2\sqrt{\rho}) = \frac{mnk}{\sqrt{M}}(1 - \sqrt{\rho})^2$$

- In other words: for 8-bit precision (assuming that the other two matrices are 64 bits), we save 41% of the memory movement.

Generalizing: Using the HBL Inequality

- This is the version of Bressan-Lieb on Wikipedia. Define $m, n \in \mathbb{N}$. Let $n_i \in \mathbb{N}, c_i \in \mathbb{R}^+$ for $i = 1, \dots, m$, and impose

$$\sum_{i=1}^m c_i n_i = n$$

- Define functions f_i integrable on $\mathbb{R}^{n_i}$ and $B_i : \mathbb{R}^n \rightarrow \mathbb{R}^{n_i}$ surjective.
- Then HBL gives

$$\int_{\mathbb{R}^n} \prod_{i=1}^m f_i(B_i x)^{c_i} dx \leq D^{-1/2} \prod_{i=1}^m \left(\int_{\mathbb{R}^{n_i}} f_i(y) dy \right)^{c_i}$$

where D is a constant that depends on the projection functions, but not the measures f_i .

- Interpretation: B_i are projections onto the faces of a hypercube, f_i are measures on the faces of the cube.

Generalizing: Using the HBL Inequality

- Again recall that A_1, A_2, A_3 be the projections of a volume element V onto the faces of the hypercube.
- We recover the standard Loomis-Whitney result for matrix multiplication by taking:
 - $n = m = 3, c_i = 1/2, n_i = 2 \forall i$,
 - $f_i = \mathbb{1}_{A_i} \forall i$, where the indicator is 1 only on the projection of the volume element.
 - Each B_i just zeros out the i 'th coordinate index and is defined only on the volume element V

Generalizing: Using the HBL Inequality

$$\int_{\mathbb{R}^n} \prod_{i=1}^m f_i(B_i x)^{c_i} dx \leq D^{-1/2} \prod_{i=1}^m \left(\int_{\mathbb{R}^{n_i}} f_i(y) dy \right)^{c_i}$$

- In our case, we want the measure of one face of the hypercube to become smaller, since that corresponds to a smaller memory footprint for that face
- The problem? We want the measure of “work” to remain the same.
- If we change $f_2 = \mathbb{1}_{V_{proj}} \rho$, notice that the factor of ρ^{c_2} shows up on both sides of the inequality, and we get no mileage.

Bounding Using Lagrange Multipliers

$$\int_{\mathbb{R}^n} \prod_{i=1}^m f_i(B_i x)^{c_i} dx \leq D^{-1/2} \prod_{i=1}^m \left(\int_{\mathbb{R}^{n_i}} f_i(y) dy \right)^{c_i}$$

- Upper-bound $f(A_1, \dots, A_m) = \prod_{i=1}^m A_i^{c_i}$ subject to the constraint $g(A_1, \dots, A_m) = \sum_{i=1}^m p_i A_i \leq M$, i.e. a weighted sum of the projection areas is less than the total fast memory size.
- Apply a Lagrange multiplier to solve the equation

$$\nabla f - \lambda \nabla g = 0$$

Let f^* be the value of f at $A_1, \dots, A_m$ satisfying the equation.

Bounding Using Lagrange Multipliers

- Compute, for $i = 1, \dots, m$,

$$(\nabla f)_i = \frac{f^* c_i}{A_i}$$

$$(\nabla g)_i = p_i$$

- Then we have $A_i = \frac{f^* c_i}{\lambda p_i}$. Plug in the constraint

$$g(A_1, \dots, A_m) = \sum_{i=1}^m p_i A_i = m$$

to get

$$\frac{f^*}{\lambda} \sum_{i=1}^m p_i \frac{c_i}{p_i} = \frac{f^*}{\lambda} \sum_{i=1}^m c_i = M$$

Bounding Using Lagrange Multipliers

- For convenience, define constant $R = \sum_{i=1}^m c_i$, and we get

$$\lambda = \frac{f^* R}{M}$$

Then $A_i = \frac{M c_i}{R p_i}$. The value of the maximum is

$$\prod_{i=1}^m A_i^{c_i} = \prod_{i=1}^m \frac{M^{c_i} c_i^{c_i}}{R^{c_i} p_i^{c_i}} \leq O \left(M^{\sum_{i=1}^m c_i} \prod_{i=1}^m p_i^{-c_i} \right)$$

- The final bound is

$$|V| \leq O \left(M^{\sum_{i=1}^m c_i} \prod_{i=1}^m p_i^{-c_i} \right)$$

Bounding Using Lagrange Multipliers

- Reasonableness check: plug in $n = m = 3$, $c_i = 1/2 \forall i$, $p_1 = p_3 = 1$, $p_2 = \rho$ into the bound above to get

$$|V| \leq O\left(\frac{M^{3/2}}{\sqrt{\rho}}\right)$$

which is the same bound that the AM-GM inequality gave us earlier.

Generalizing the Lower Bound: Work Segments

- To generalize our lower bound from mixed precision matrix multiplication to the dense-times-random case, we now make the distinction between two types of work:
 - Computational work (CW): addition and multiplication
 - Loading work (LW): anything we do to get operands into the cache to carry out computational work; this can be recomputation or the loading of lower precision words into the cache
- Now split the computation into phases where we can perform at most T units of loading work in each segment; we can always find such a division.

Generalizing the Lower Bound: Work Segments

- Our new goal: maximize the amount of computational work V with T units of loading work.
- Suppose moving a word of either A or C into the cache costs us a single unit of LW, while loading a word of B costs ρ units of LW.

Converting Theory to Practice

- For software GEMM implementations, tiling sizes are affected by factors other than optimal cache reuse, which are highly architecture-specific:
 - Loop unrolling
 - Data layout and vector register width (e.g. AVX256 vs. AVX512)
 - Restricting tile size to powers of 2, combined with cache capacity constraints
 - Software Prefetching (boost of more than 20%, in our case!)
- In practice, standard GEMM tile shapes tend to be rectangular for the reasons above. Our theory won't predict the exact tiling shape for software implementations, but it will predict that the optimal tiling skews more and more rectangular as $\rho \rightarrow 0$.
- More on hardware GEMM implementations later...

Experiment 1: Artificial Time Delay, Take 2

```
//col = _mm512_load_pd(B + 0);  
  
for(int i = 0; i < DELAY_CYCLES; i++) {  
    value += 1.4834;  
}  
  
Ccol0_0 = _mm512_fmadd_pd(_mm512_set1_pd(A[0]), col, Ccol0_0);  
Ccol0_1 = _mm512_fmadd_pd(_mm512_set1_pd(A[1]), col, Ccol0_1);  
Ccol0_2 = _mm512_fmadd_pd(_mm512_set1_pd(A[2]), col, Ccol0_2);  
Ccol0_3 = _mm512_fmadd_pd(_mm512_set1_pd(A[3]), col, Ccol0_3);  
Ccol0_4 = _mm512_fmadd_pd(_mm512_set1_pd(A[4]), col, Ccol0_4);  
Ccol0_5 = _mm512_fmadd_pd(_mm512_set1_pd(A[5]), col, Ccol0_5);
```

Figure 2: Inserting Dummy FLOPS

- We are now comparing against an optimized GEMM implementation that performs at 86% of the performance of the Vendor MKL library (our previous homebrewed implementation got less than 1/2 of the MKL performance).
- Most of the overhead comes from packing / transposing the matrices.
- Micro-kernels are auto-generated using Cog (working on integrating Cog into the build system)

Tile Shape Effect on Performance

- 120×480 , A stationary, supposedly optimal for standard GEMM.

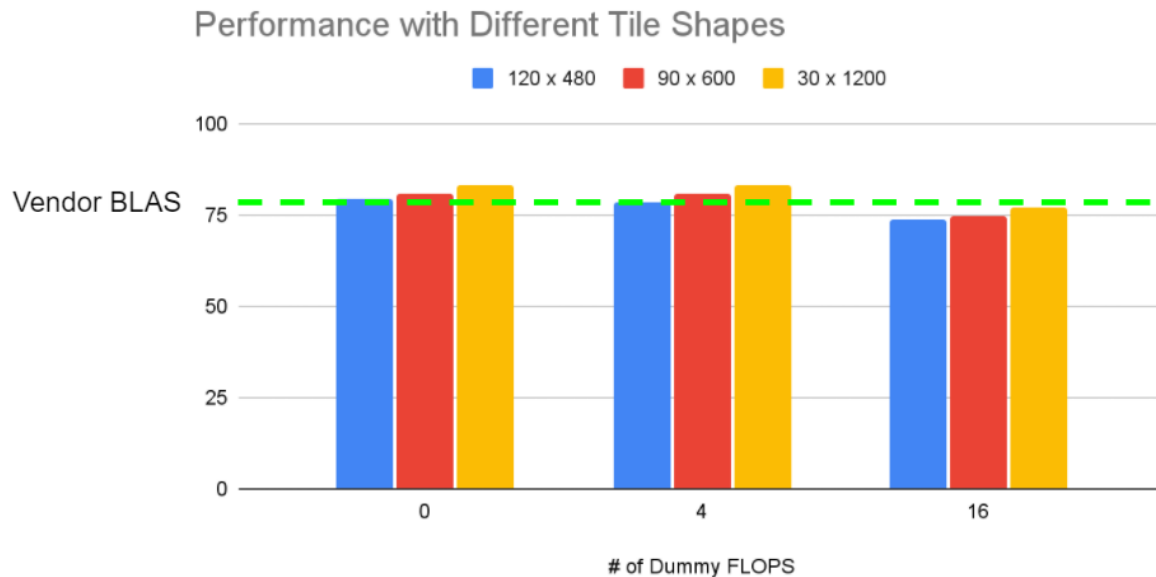


Figure 3: Effect of Tiling Shape on Performance

Standard GEMM Tiling Shape Strangeness...

Standard GEMM Tiling Shape	Percent of Peak
120 x 480	69.98
90 x 600	70.7
30 x 1200	70.95

Figure 4: Standard GEMM Shape Table

Strange Timing Behavior Explained

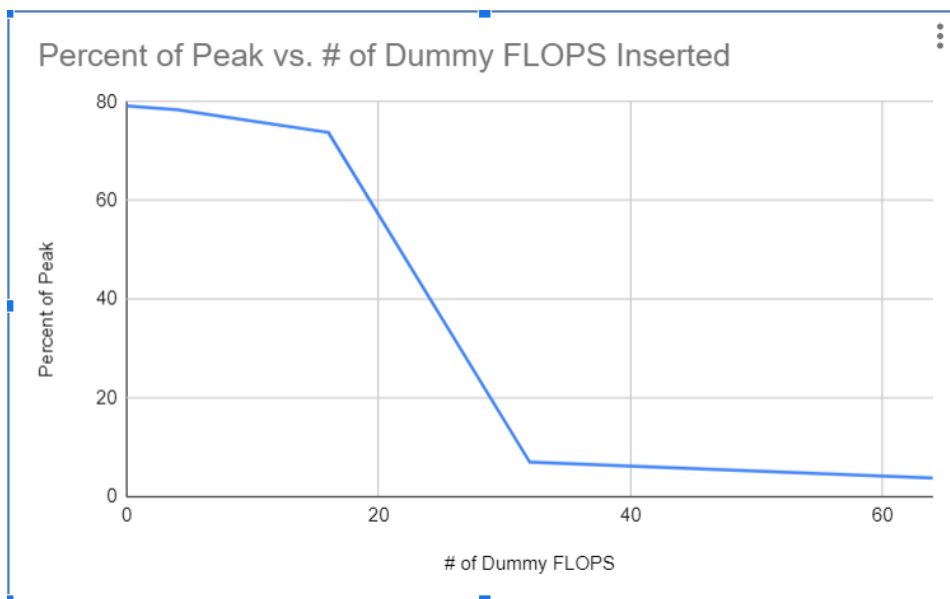


Figure 5: Performance Impact of Dummy FLOPS

Problems with this Experiment

- KNL Cache is direct-mapped, and not explicitly managed! In other words, it is likely that useful data gets evicted from the cache when it could potentially be reused.
- Inserting dummy FLOPs to simulate a time delay is perilous...
 - Compiler won't optimize away the dummy flops, but it can judge that they won't affect the rest of the computation and potentially perform out-of-order scheduling.
 - Chart above shows nonlinear scaling behavior with the number of dummy FLOPS that we insert

Using GEMMINI Instead

- Why is GEMMINI the right tool for the job?
 - Granular control over data flow
 - The predicted tile shape results in a rectangular systolic array, with the dimensions depending on the cost of the random number generator.
 - Less obfuscation of the main result by software / hardware quirks
 - Can integrate with the hardware random number generators that Jingyi is developing.

The Road to Using GEMMINI

- Weeks 1-2: GEMMINI Installation and Introductory Lab. Just implement standard GEMM. Need to pick up Verilog and Scala.
 - I know how to use ABEL, which is a really archaic HDL, but nothing modern. Not starting from scratch, but pretty close.
 - I'm more familiar with Scala, so that's less of an issue.
- Week 3-4: Modify the standard GEMM implementation to use a rectangular tile instead of a square tile, and use some kind of dummy operation to simulate random number generation. Also, can see if there's a way to simulate mixed precision arithmetic.
- Probably this will involve collaborating with Jingyi; I'll wait until I get the lab set up and working before I bug her.

- Some more theory on the third potential application of the asymmetric access cost theory
- More testing in software and benchmarks
- More details on GEMMINI

Application 3: One Matrix Higher Cost than the Other

- Take the third example on the first slide, where we attempt to accumulate a result in a higher precision than the two input matrices (which GEMMINI actually does)
- We can reduce the memory movement cost using the theory above... but only if we keep one of the two input matrices stationary (with a single level of caching).
- This is not ideal; we should keep the (heavier) output matrix stationary instead to avoid any cubic term in moving it at all.

The Solution: GoToBLAS!

- With multiple levels of caching, we find a good use for the theory again.
- GoToBLAS: Blocking optimized for multiple levels of caching. **At each level of blocking, one matrix stays stationary while the other two matrices stream in.**
- Corollary that arises if we actually try to implement this: if we choose C to be stationary at the L3 level, we can no longer select C to be stationary at the L2 level; we must choose either A or B .
- So we are again forced to choose rectangular tiles for optimal performance, but just at a different level of blocking.

More Testing in Software

- Tried the simplest possible PRNG I could find, an XORShift (which does not pass many statistical tests).

```
//_mm_prefetch(B + NR * 28, _MM_HINT_T0);  
//col = _mm512_load_pd(B + 0);  
icol = _mm512_xor_epi64(_mm512_bslli_epi128(icol, 7), icol);  
icol = _mm512_xor_epi64(_mm512_bslli_epi128(icol, 9), icol);  
icol = _mm512_xor_epi64(_mm512_bslli_epi128(icol, 13), icol);  
col = _mm512_cvtepu64_pd(icol);
```

Figure 6: A (Close Enough) XORShift

- The problem: A performance drop to about 52% of peak as soon as these instructions get put in
- At this level, replacing a prefetched load with 7 vector instructions that must execute in sequence.

Progress on Learning GEMMINI

- Met with Alon; here are some of my notes from that meeting
- GEMMINI is not just a systolic array that does 8×8 or 32×32 matrix multiplications (although that is the core). The complicated part is synchronized timing to move data in from the L2 to scratchpad and from scratchpad into the processing elements to keep the systolic array busy.
- Changing the shape of the smallest systolic array to a rectangle is a bad idea; since data needs to be staged in the array before computation begins, a rectangular systolic array will introduce a “bubble” that delays the staging of one of the two inputs along the longer dimension of the rectangle.

The GEMMINI Architecture

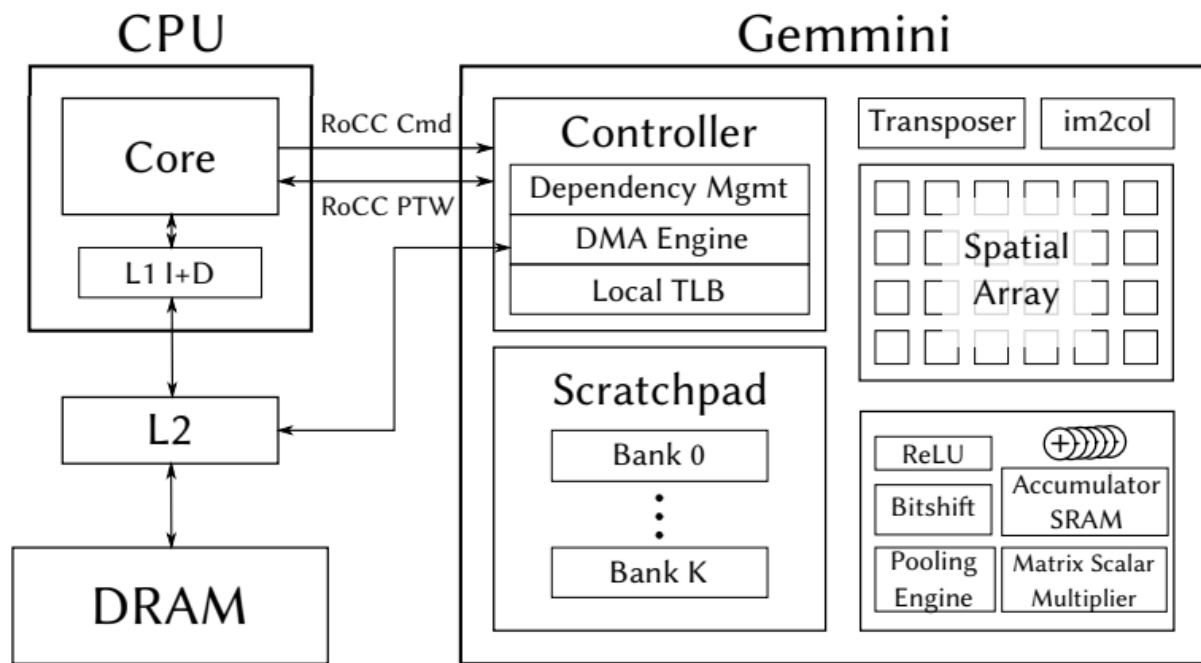


Figure 7: GEMMINI Architecture

Progress on GEMMINI

- On the other hand, I can (allegedly) tell GEMMINI that we want to generate the hardware instructions to multiply an 256×64 matrix, and it will do that by issuing several calls to the micro-kernel. At this higher level, we can test the predictive model of tile shape
- GEMMINI already accumulates its result in a higher precision than the input. If we choose a weight stationary algorithm, our theory should be able to predict the optimal tile size

- Suggestion from Alon: Collaborate with both Jingyi and Dima, who have already made some progress with implementing random number generators in hardware
- For the mixed precision experiments, there is some minor effort involved in changing the data types of the input operands, which will then generate the correct hardware design
- For the dense-times random experiments, it gets more involved; Alon suggested replacing the transposer module in GEMMINI to feed in random numbers instead of data from scratchpad

- Replaced dummy FLOPs with an XORShift RNG.
- Realized we were regenerating the random numbers at the “micro-kernel level”, not at the L2 level like we wanted
- The screenshot above where we replace the load operation with the dummy FLOPS means that the random numbers only get reuse at the micro-kernel tile level, but we want them to get reuse at the L2 level (where we’re modifying the tile shape).
- Simple fix: set up a small region of memory (that we access enough that it’s guaranteed will stay in L2) to cache random numbers. We’re still regenerating, but we at the L2 level, no longer at the register level)

Using an L2 Buffer for Random Numbers

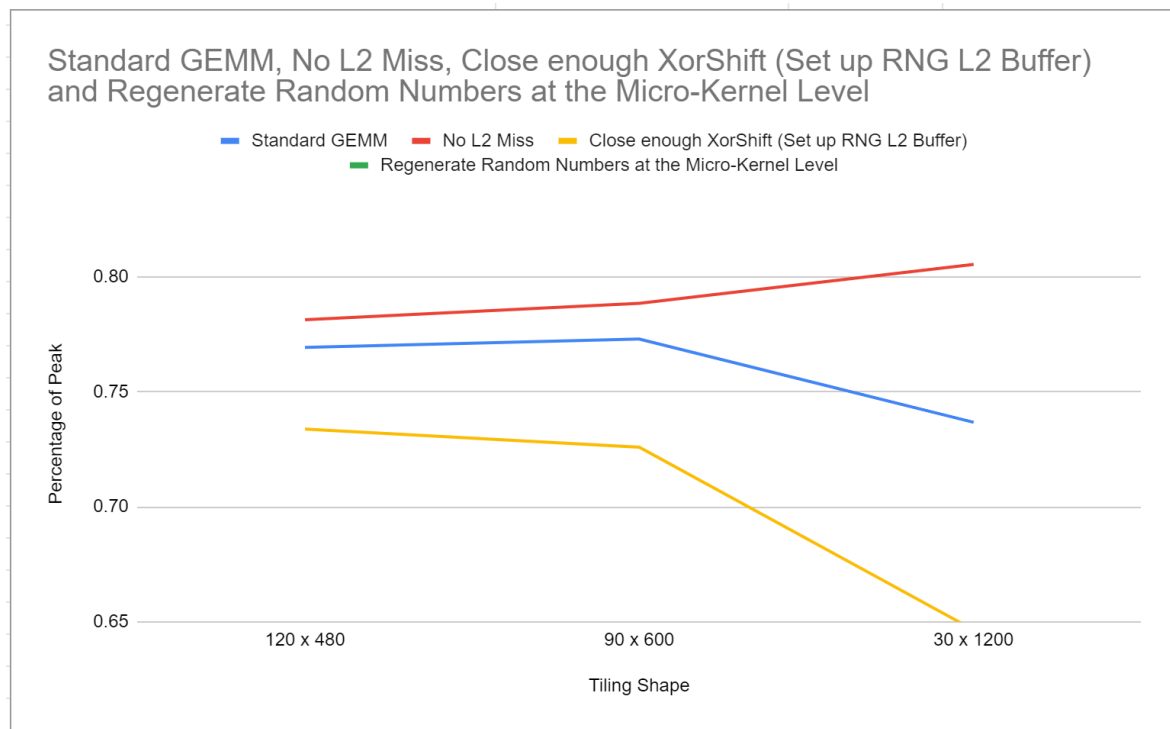


Figure 8: Experiments Testing L2 Buffer for Random Numbers

- See the whiteboard for definitions
- Increasing KB allows more runs of the loop calling the micro-kernel, leading to a mild boost in efficiency
- On the other hand, tiles that are too rectangular (for standard GEMM) cause L2 misses, which is why the blue line dips

- What if we magically had no L2 misses? Then there is no performance drop when we choose very rectangular tiles, and in fact a performance increase (as our theory predicts)
- What happens when we use an XoRShift to fill up an L2 random number buffer? Consistent performance drops as the tile gets more rectangular
- Regeneration at the micro-kernel level: see the spreadsheet (performance hovers around 50%).

Takeaways

- Our tile shape experiments show that our RNG is slower than the DRAM to L2 transfer time.
- The takeaway: software random number generation is difficult, since state-based RNGs have a non-trivial critical path. Peak FLOP performance depends on
 - Pipelining instructions
 - Simultaneously using multiple vector processing units (Intel calls them lanes) on a processor
 - Prefetching to hide latency
- Random number generation can take advantage of all of these things, but much less effectively than the matrix multiplication micro-kernel

July 27 Progress

- Spent last week collaborating with Jingyi on random number generators and the randomized SVD, as well as GEMMINI and Systol
- Today: some notes and takeaways from that collaboration

Some Notes on Hardware Applications for Random Number Generators

- I think it would help to show a runtime comparison of the Halko SVD and, say, CUR (skeleton) decomposition along with the usual SVD runtime, since I don't believe the Apple folks understand how good the scaling is if you only want a few singular values
- For the Halko SVD, Jingyi and I were somewhat stuck on the point that random number generation computes only a lower-order term to the computational cost
- I'm not sure about the procedure for CUR Skeleton decomposition (the one where you randomly sample rows and columns, trying to maximize the determinant of the cross matrix), but my guess is that the randomness isn't the high-order term

Brainstorming Uses for the Hardware RNG

- We can integrate into dense-times-random, but that's going to require low-level GEMMINI integration (more work still needed)
- I suggested random walks, particle simulations, Monte Carlo simulations, maybe even raytracing (??) as applications where randomness might be the higher-order term
- Presumably, Apple wants to provide its game developers with tools that can do all of these things, so it's potentially important.

Singular Value Spectra of Neural Network Layers

- Jingyi said she may (depending on the time she has) reach out to Arissa about the singular value spectrum of the trained neural network matrices.
- Suggestion to her: It may also be useful to load in SOTA famous neural network (i.e. deep LeNets, Alexnets) and hit their fully-connected layers with an SVD. Can do this very easily in Matlab, as it turns out
- Dense-Times-Random might have some applications in physical science domains